UdeA
Analytics &
DataScience

DATA STREAMING Y SERVICIOS EN LA NUBE

Procesamiento de Datos

Magister - Efraín Alberto Oviedo
alberto.oviedo@udea.edu.co

UNIVERSIDAD DE ANTIOQUIA
FACULTAD DE INGENIERÍA
ESPECIALIZACIÓN EN ANALÍTICA Y CIENCIA DE DATOS

UNIVERSIDAD DE ANTIOQUIA
1 8 0 3

AGENDA

UdeA
Analytics & DataScience

1. **Procesamiento Basado en Eventos**
2. Batch Processing
3. Continuos Stream Processing
4. Servicios de Stream Processing
   - Apache Kafka
   - Apache Flink

Procesamiento Basado en Eventos

Sistemas Informáticos Tradicionales (Batch Processing)

- Ejecuciones nocturnas
- Tareas Programadas

Mundo Actual

- Millones de transacciones ocurren cada segundo
- Sensores generan datos de forma continua
- Aplicaciones reaccionan instantáneamente a los usuarios
- Las decisiones deben tomarse en milisegundos.

Eventos

Un **evento** es una representación digital de algo que **ya ocurrió** en el mundo real o en un sistema

Es una **señal inmutable** que indica que algo pasó.

Ejemplos cotidianos de eventos

- Un usuario hace clic en un botón.
- Una tarjeta de crédito es usada en una tienda.
- Un sensor registra una temperatura.
- Un sistema genera un mensaje de error.
- Un archivo es subido a la nube.

Componentes de un Evento

- **Identidad del evento:** Describe qué ocurrió.

  Compra realizada, sensor activado, login fallido

- **Datos asociados**: Describen el contexto del evento (Atributos)

  usuario, monto, ubicación, tipo de dispositivo, valor medido.

- **Tiempo**: Todo evento está ligado al tiempo. Existen dos nociones importantes:
  - **Tiempo de evento**: momento real en que ocurrió el hecho.
  - **Tiempo de procesamiento**: momento en que el sistema lo procesa.

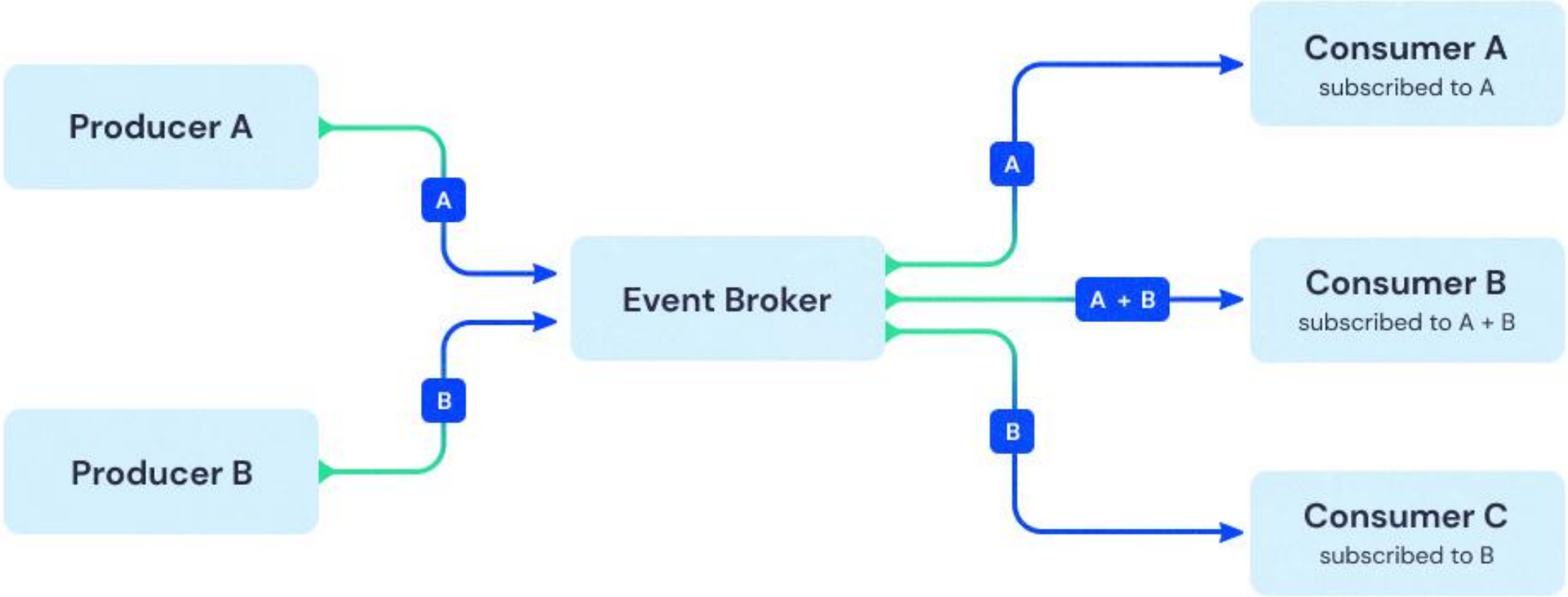
Ejemplo de un Evento


```
{
  "tipo": "CompraRealizada",
  "usuario_id": 1245,
  "monto": 350.00,
  "moneda": "USD",
  "timestamp_evento": "2025-03-10T15:42:30Z"
}
```


Este evento representa una **verdad histórica**: la compra ya sucedió y no puede cambiarse.

Arquitectura Basada en Eventos

Todo sistema orientado a eventos puede explicarse con tres bloques fundamentales



https://www.adservio.fr/post/event-driven-architecture

Productor (Producer)

El **productor** es el componente que **genera eventos**.

- No decide quién los procesa.
- No espera respuestas.
- Simplemente **emite eventos**.

Ejemplos reales:

- Una aplicación web
- Un microservicio
- Un sensor IoT
- Un sistema de logs
- Un dispositivo móvil

Bus de Eventos (Event Broker)

Es el **corazón del sistema basado en eventos**

- Recibir eventos
- Almacenarlos temporalmente
- Entregarlos a uno o varios consumidores
- Desacoplar productores y consumidores

Ejemplos reales:

- AWS Kinesis Data Streams
- Apache Kafka
- Azure Event Hubs
- Google Pub/Sub

Consumidor (Consumer)

El **consumidor** es el componente que **reacciona al evento**.

- Procesar datos
- Generar alertas
- Almacenar información
- Actualizar dashboards
- Activar otros sistemas

Un mismo evento puede ser consumido por múltiples consumidores, cada uno con un propósito diferente.

Desacoplamiento

La gran ventaja del enfoque basado en eventos

Una de las ideas más importantes del procesamiento basado en eventos es el **desacoplamiento**.

- El productor **no conoce** al consumidor.
- El consumidor **no controla** al productor.
- Ambos trabajan de forma independiente.

Taller Comprendiendo los Eventos

Una empresa está desarrollando una plataforma que recibe información de múltiples fuentes: usuarios, sensores, aplicaciones móviles y sistemas externos.

La empresa quiere evolucionar hacia un sistema que reaccione en tiempo real, sea escalable y pueda adaptarse a nuevas fuentes de datos sin reescribir todo el sistema.

- Un usuario inicia sesión en una aplicación.
- Un sensor reporta una temperatura cada segundo.
- Un cliente realiza una compra.
- Una tarjeta es rechazada por posible fraude.
- Un archivo es subido a un sistema.
- Un reporte mensual es generado.

AGENDA

UdeA
Analytics & DataScience

1. Procesamiento Basado en Eventos
2. **Batch Processing**
3. Continuos Stream Processing
4. Servicios de Stream Processing
   - Apache Kafka
   - Apache Flink

Batch Processing

El **Batch Processing** (procesamiento por lotes) es un paradigma en el cual:

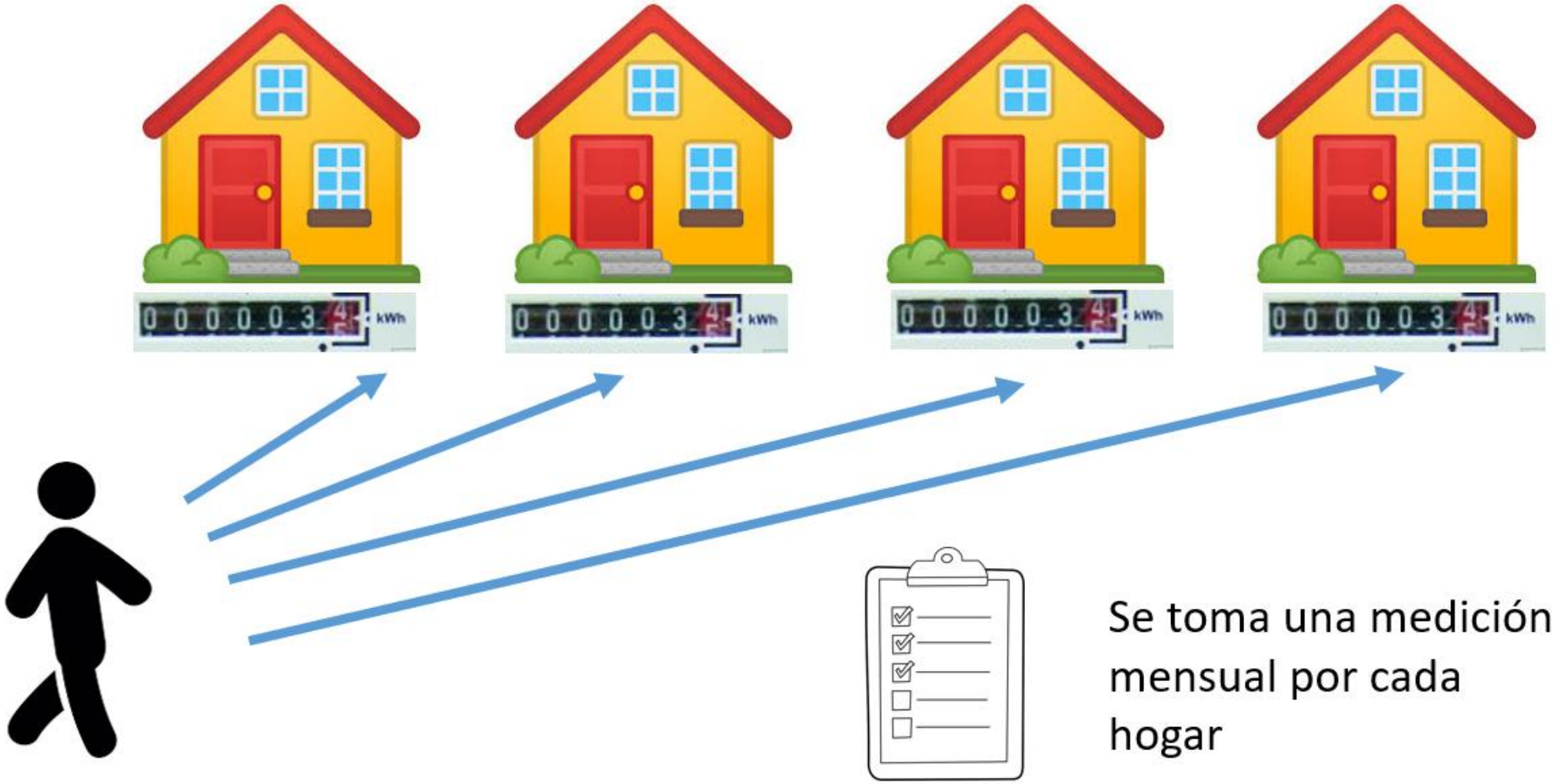
- Los datos se **acumulan durante un período de tiempo**.
- Se almacenan de forma persistente.
- Un proceso se ejecuta para **analizar todos esos datos juntos**.

Este enfoque responde a la pregunta:

“¿Qué ocurrió durante un período de tiempo determinado?”

En lugar de reaccionar a cada evento individual, el sistema **espera**, agrupa la información y luego la procesa en bloque.

Ejemplo: Facturación de servicios Públicos



Se toma una medición mensual por cada hogar

Características

- **Procesamiento periódico:** El batch se ejecuta en intervalos definidos diario, semanal, mensual o bajo demanda. No está activo permanentemente.
- **Datos finitos por ejecución:** Cada ejecución trabaja con un conjunto cerrado de datos claramente delimitado en el tiempo. Ejemplo: “Procesar todas las ventas del día de ayer”.
- **Alta eficiencia para grandes volúmenes:** Al procesar datos en bloque se optimiza el uso de CPU, se reduce el overhead y se minimizan costos. Por eso el batch es ideal para **Big Data histórico**.
- **Latencia elevada:** Entre el momento en que ocurre un evento y el momento en que se analiza, puede pasar minutos, horas o días. Esto es aceptable cuando **el tiempo no es crítico**.

Ventajas

Estas razones explican por qué **batch sigue siendo ampliamente utilizado**.

- Simplicidad conceptual.
- Costos más bajos.
- Alta eficiencia para grandes volúmenes.
- Facilidad de auditoría.
- Reprocesamiento sencillo.

Limitaciones

Batch Processing **no es la solución universal**, sino una herramienta más.

- No permite reacción inmediata.
- Información desactualizada.
- No apto para alertas críticas.
- Poco flexible ante eventos inesperados.

Notebook: 03_Simulación Batch

Batch Processing

Simulación de *ventas diarias* y generación de reporte **batch**

Vamos a comprender de forma práctica, cómo funciona el **Batch Processing** usando un escenario cotidiano:

> **Una tienda registra ventas durante el día** y al final genera un **reporte consolidado** (batch).

¿Qué aprenderás aquí?

- Qué significa **acumular datos** antes de procesarlos.
- Por qué el batch tiene **latencia alta** y por qué, en muchos casos, eso es **aceptable**.
- Cómo un proceso batch produce **resultados agregados** (totales, promedios, top productos) en vez de reaccionar evento a evento.

AGENDA

UdeA
Analytics & DataScience

1. Procesamiento Basado en Eventos
2. Batch Processing
3. **Continuos Stream Processing**
4. Servicios de Stream Processing
   - Apache Kafka
   - Apache Flink

Continuous Stream Processing

El **Continuous Stream Processing** (procesamiento continuo de flujos) es un paradigma en el cual:

- Los datos llegan como un **flujo continuo de eventos**
- Cada evento se procesa **tan pronto como es recibido**
- El sistema está siempre activo y reaccionando

Este enfoque responde a la pregunta: **“¿Qué está ocurriendo ahora mismo?”**

A diferencia del batch, no existe un “final del período”. El sistema **nunca deja de procesar**.

El evento: unidad central del streaming

En streaming, la unidad fundamental **no es el archivo ni el lote**, sino el **evento**. Un evento representa algo que ocurrió en un momento específico con información asociada.

```
{
  "sensor_id": "sensor-12",
  "temperatura": 82.5,
  "timestamp": "2025-03-10T14:21:45Z"
}
```

Cada evento:

- Se procesa de forma individual o incremental
- Puede generar una acción inmediata
- Puede alimentar métricas en tiempo real

Características

- **Procesamiento en tiempo (casi) real:** Los eventos se procesan en milisegundos o segundos, apenas llegan al sistema.
- **Flujo infinito de datos:** El stream no tiene un tamaño finito, no hay un “conjunto completo” de datos, el sistema debe operar indefinidamente
- **Procesamiento incremental:** Los resultados se actualizan evento por evento, sin reprocesar todo el histórico. Ejemplos: Contador que aumenta, Promedio que se recalcula, Estado que se actualiza
- **Arquitectura reactiva:** El sistema no consulta si hay datos, **reacciona** cuando llegan.

Arquitectura típica

Una arquitectura de streaming suele incluir:

- **Productores de eventos:** sensores, aplicaciones, microservicios o sistemas externos.

- **Cola o plataforma de streaming:** recibe eventos, los ordena, los distribuye y desacopla productores y consumidores.

- **Consumidores de eventos:** procesan eventos, generan acciones, actualizan estados, almacenan resultados.

Ventajas

Cuando utilizamos el Stream Processing tenemos las siguientes ventajas:

- Reacción inmediata.
- Mayor control en sistemas críticos.
- Experiencia de usuario mejorada.
- Sistemas más inteligentes.
- Capacidad de actuar antes de que el problema crezca.

Por estas razones, el streaming es clave en sistemas modernos.

Desafíos

El streaming no es trivial, implica retos importantes como los mencionados a continuación:

- Mayor complejidad arquitectónica.
- Manejo del tiempo y del orden de eventos.
- Posibles eventos tardíos.
- Costos más altos.
- Necesidad de escalabilidad dinámica.

Por eso **no todo debe ser streaming**.

Notebook: 04_Simulación Streaming

Notebook demostrativo: Productor y Consumidor con Python y `asyncio`

Aquí vamos a construir, paso a paso, una **simulación sencilla de streaming de datos** usando solo Python:

- Primero veremos un **escenario ideal sin cola**, donde el productor llama directamente al consumidor (**escenario 1**).
- Luego construiremos escenarios con **cola de mensajes** usando `asyncio.Queue` para simular:
  - Un caso **saludable**: productor y consumidor trabajan a ritmos similares (**escenario 2A**).
  - Un caso con **acumulación (backlog)** cuando el consumidor es más lento (**escenario 2B**).
  - Un caso más cercano a **batch**, donde procesamos los eventos en ventanas de tiempo (**escenario 3**).

Taller Batch y Streaming

Hemos estudiado dos paradigmas fundamentales para el procesamiento de datos:

- **Batch Processing**: basado en la acumulación y análisis posterior de los datos.
- **Continuous Stream Processing**: basado en la reacción inmediata a eventos que ocurren continuamente.

Ambos enfoques **no compiten entre sí**. Cada uno responde a **necesidades distintas**, y entender cuándo usar uno u otro es una **habilidad clave de diseño de sistemas de datos**.

AGENDA

UdeA
Analytics & DataScience

1. Procesamiento Basado en Eventos
2. Batch Processing
3. Continuos Stream Processing
4. **Servicios de Stream Processing**
   - **Apache Kafka**
   - Apache Flink

Apache Kafka

Apache Kafka es una de las plataformas de streaming más utilizadas en la industria.

- Plataforma distribuida de eventos.
- Almacenamiento de streams en **topics**.
- Escalabilidad horizontal.
- Alta durabilidad y tolerancia a fallos.
- Relectura de eventos.

Kafka **no procesa** eventos por sí mismo, sino que:

- Actúa como **columna vertebral del stream**.
- Desacopla productores y consumidores.
- Almacena eventos para procesamiento posterior.

Opciones para utilizar Kafka

- **Kafka Local (un solo PC):** Kafka corriendo en tu máquina (Linux/Windows/Mac).
- **Kafka en Contenedores:** Docker / Docker Compose/ Kubernetes
- **Kafka Autogestionado en Servidores (VMs)**
- **Kafka como servicio grestionado:** Un proveedor te entrega Kafka listo para ser usado
  - **Confluent Cloud**

Crear cuenta en Confluent Cloud

1. Entra a la página de “Get started” de Confluent y crea tu cuenta (permite SSO con Google/GitHub o email).
   https://www.confluent.io/get-started/

2. Confirma/valida el email si te lo solicita.

3. Entra a la **Confluent Cloud Console**.

Resultado: tienes acceso al panel donde crear clusters, topics y credenciales.

Crear un cluster Kafka

1. En la consola, haz clic en **Add cluster**.
2. Selecciona un tipo de cluster (para laboratorio suele bastar **Basic**/serverless según disponibilidad en tu cuenta).
3. Elige el **cloud provider** y la **región** (escoge la región más cercana a tu audiencia para menor latencia).
4. Crea el cluster y espera a que esté en estado “running/ready”.

Con esto ya tienes el “Kafka remoto” funcionando.

Obtener el “Bootstrap Server”

- En tu cluster, abre **Cluster Settings** y copia el **Bootstrap server**.
- Este valor se ve como hostname:port y es el endpoint que puedes usar para conectarte con tu servidor de kafka

Crear credenciales: API Key / Secret

Para utilizar Kafka en Confluent Cloud normalmente se accede con **SASL/SSL** usando:

- **API Key** como “username”
- **API Secret** como “password”

Pasos recomendados:

1. Ve a la sección **API keys** y crea una nueva key.
2. Idealmente crea credenciales para un **service account** (buena práctica para workloads, aunque para laboratorio puede bastar con key asociada al usuario).
3. Copia y guarda **Key** y **Secret** (una vez creada, el secret puede no volver a mostrarse igual).

Crear el Topic (canal de eventos)

Crea un topic desde la consola

- Topic: sensores_iot
- Particiones: 3
- Retención: por defecto

Notebook: 05_Kafka_ConfluentCloud

Apache Kafka con Confluent Cloud

Notebook paso a paso: producir y consumir eventos

En este laboratorio usarás **Kafka como cliente** desde Google Colab para conectarte a un **cluster Kafka gestionado** (Confluent Cloud).

Al final podrás comprender:

- Qué es un **topic** y qué significa que tenga **particiones**
- Qué es una **key** (clave) y cómo influye en la **partición**
- Qué es un **offset** y cómo Kafka recuerda el progreso de lectura
- Qué es un **consumer group** y por qué es importante en streaming

> **Idea clave**: Kafka no es una "librería". Kafka es un **servicio (broker/cluster)** que vive en Confluent Cloud.
> En Colab solo instalamos el **cliente** para conectarnos a ese servicio.

AGENDA

UdeA
Analytics & DataScience

1. Procesamiento Basado en Eventos
2. Batch Processing
3. Continuos Stream Processing
4. **Servicios de Stream Processing**
   - Apache Kafka
   - **Apache Flink**

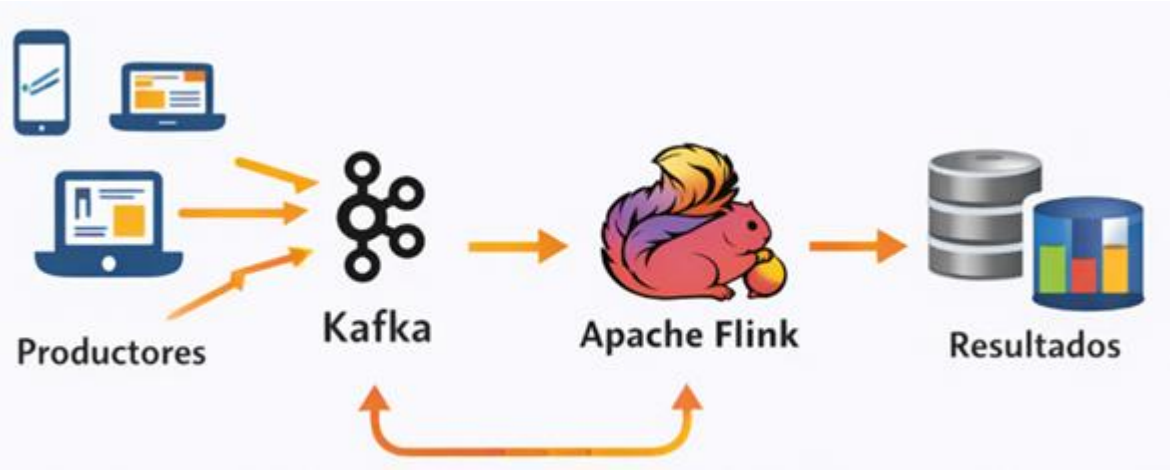
Apache Flink

Motor de procesamiento de streams, diseñado para trabajar **evento a evento**, con baja latencia.

- Streaming nativo (no micro-batch).
- Manejo avanzado de **estado**.
- Tolerancia a fallos.
- Procesamiento exactamente una vez.

En arquitecturas típicas:

- Kafka provee los eventos.
- Flink los procesa en tiempo real.



Kafka + Flink es una combinación muy común en sistemas modernos.

Flink SQL

Flink SQL es una capa declarativa sobre Apache Flink que permite:

- Definir **streams como tablas**
- Ejecutar **consultas continuas**
- Trabajar con **ventanas de tiempo**
- Procesar datos **en tiempo real**

En Flink SQL, una consulta **no termina**, se mantiene activa mientras llegan eventos.

Apache Flink SQL en Confluent Cloud

1. Accede a Confluent Cloud
2. Selecciona el **Environment**
3. Selecciona el **Kafka Cluster** usado en el curso
4. Accede a Flink e ingresa a la pestaña Compute pools
5. Has clic en Crear Compute Pool
   - Cloud provider: el mismo donde está el Kafka cluster (recomendado)
   - Region: la misma región del Kafka cluster (recomendado)
   - En tamaño/capacidad, selecciona el **mínimo** o **default**
   - Crea el compute Pool
6. Has clic en Open SQL Workspace: Editor de SQL donde se ejecutarán las consultas

Apache Flink SQL en Confluent Cloud

Primera consulta: ver el stream crudo

Observar cómo Kafka expone los eventos sin imponer esquema.

Explora la tabla

- SELECT * FROM sensores_iot

Taller Apache Flink

El objetivo de esta demostración es observar en tiempo real cómo los eventos producidos en Kafka son procesados continuamente por Apache Flink y generan resultados que cambian dinámicamente a medida que llegan nuevos datos

Todo esto sin desplegar Flink, sin escribir código Java/Python, y usando servicios gestionados.